NatureMatch — proposed Scrum development plan

Product goal. Help someone decide whether a monitored green space suits their intended activity, using measured environmental conditions, estimated occupancy and recorded access information.

The development approach combines separate modules with complete, working increments. Each sprint extends the system from physical sensing through to useful feedback.

1. Agree on a realistic MVP

The committed scope is:

- One Raspberry Pi station monitoring one controlled entrance.
- Entry/exit estimation using two infrared detection points.
- Temperature/humidity, noise and light measurements, subject to hardware verification.
- LED feedback linked to measured conditions or occupancy.
- Timestamped JSON records, persistent storage and a Grafana dashboard.
- A simple web interface with two activity profiles: **quiet studying** and **socialising**.
- Recorded access requirements checked before activity suitability is calculated.
- Visible measurement freshness, missing data and sensor limitations.
- Installation instructions, OpenAPI documentation, logs and basic Raspberry Pi health reporting.

With one physical station, the application will assess the suitability of **one monitored place**. Tests can use explicitly labelled example places to demonstrate filtering and ranking.

Optional backlog: NFC sessions, UV and air-quality sensors, additional stations, external APIs, maps and MQTT. LoRaWAN remains a future extension.

This scope assumes the group can reuse working lab code and the existing HTTP server. Hardware availability and compatibility must be checked in Sprint 1 before confirming the sensor selection.

2. Keep the modules independently understandable

Module	Responsibility	Boundary
Station	Read sensors and interpret infrared crossing sequences	Produces observations and occupancy estimates
Shared data model	Define timestamps, identifiers, units and quality states	Used consistently across the system

Service and storage	HTTP/JSON API, persistence and historical queries	Connects station data to consumers
Activity assessment	Apply access requirements, then activity-specific rules	Produces suitability and understandable reasons
User interface and dashboard	Show current conditions, assessments and history	Consumes the API and stored measurements
Operations	Startup, configuration, logs, health and installation	Makes the prototype reproducible

Sensor-specific code stays inside the station module. Activity rules consume normalised measurements, allowing the group to test them without occupying the Raspberry Pi.

Estimated occupancy and voluntary NFC registrations remain separate if NFC is added later.

3. Use the official sprint dates

Sprint 1 officially began on **3 November**. The remaining capacity below is calculated from **5 November**; record work on its actual date.

All estimates are **total team person-hours**, including the time of both participants when pairing.

Sprint	Official dates	Planned work	Coordination	Contingency
1	3–17 November	14 h	4 h	3 h
2	17 November–3 December	18 h	4 h	5 h
3	3–11 December	8 h	2 h	4 h
Total		**40 h**	**10 h**	**12 h**

These are initial forecasts. Re-estimate at each Sprint Planning using actual progress and remaining availability.

Sprint 1 — prove the station and establish the project

Sprint Goal: demonstrate that the selected hardware works and produces a trustworthy local status.

Owner	Work package	Estimate
Aimar	Infrared wiring and crossing logic; manual comparison tests; repository contribution workflow	5 h
Alessandro	Environmental sensor checks; interface/pin inventory; shared JSON record format	4.5 h
Leire	LED feedback; minimal HTTP status view; GitHub Projects setup and prioritised backlog	4.5 h

Internal checkpoints:

- **By 10 November:** agree sensor selection, wiring, data format and module boundaries.
- **By 16 November:** integrate counting, environmental readings and LED feedback; rehearse the review.

Review on 17 November: show a live local increment, the architecture, tested sensors and actuator, assigned issues, contribution history and the next sprint's backlog.

Acceptance evidence:

- Controlled entries and exits are compared with manual observations.
- Pauses, reversals and incomplete crossings have explicit handling.
- Selected sensors produce timestamped readings.
- A real condition change triggers the LED.
- Each member has contributed reviewed work from their own account.

Sprint 2 — turn measurements into useful information

Sprint Goal: demonstrate a complete live flow from physical sensing to storage, visualisation and an initial activity assessment.

Owner	Work package	Estimate
Aimar	Integrate station acquisition; improve counting; local logging/recovery; startup and health reporting	5.5 h
Alessandro	InfluxDB persistence; API endpoints; OpenAPI documentation; historical query or simple summary	6 h
Leire	Web interface; surveyed place information; two activity profiles; Grafana panels	6.5 h

Internal checkpoints:

- **By 24 November:** one real sensor feeds storage and a visible dashboard.
- **By 30 November:** occupancy and selected environmental readings reach the interface; activity assessment works.
- **By 2 December:** rehearse the complete live demonstration and resolve integration failures.

Acceptance evidence:

- A physical change appears in persistent storage and the interface.
- Stored history supports a simple average, median or trend.
- Quiet studying and socialising produce different assessments where appropriate.
- Essential access requirements are checked before preferences.
- Missing or stale measurements are visible.
- LED behaviour has a documented relationship to the data.
- All three members have implementation commits and have reviewed another member's work.

Review on 3 December: demonstrate the live flow and inspect the board with the teacher. Agree the final delivery scope at this checkpoint.

Sprint 3 — validate and deliver

Sprint Goal: deliver a reproducible, tested MVP that all three members can explain.

Owner	Work package	Estimate
Aimar	Counting validation, restart/recovery checks and station defect fixes	2.5 h
Alessandro	Clean installation check, API documentation and release preparation	3 h
Leire	Interface/data-quality checks, README/wiki completion and submission packaging	2.5 h

All three contribute to the joint reflection and demonstration rehearsal.

Internal checkpoints:

- **By 7 December:** complete controlled validation and a supervised field test where practical.
- **By 10 December:** complete documentation, rehearse and freeze the release candidate.
- **11 December:** tag the evaluated version, publish the release and submit through ALUD.

Only pull optional work into this sprint when the core meets the Definition of Done and delivery time remains protected.

4. Share Scrum responsibilities and technical knowledge

The following rotation follows the proposal's intention to share management responsibilities. These are additional accountabilities: **everyone remains a developer**.

Sprint	Product Owner	Scrum Master
1	Aimar	Leire
2	Alessandro	Aimar
3	Leire	Alessandro

The Product Owner keeps priorities and acceptance criteria clear. The Scrum Master facilitates events and helps remove blockers. Hold a short handover when these responsibilities rotate.

Use these lightweight events:

Event	Working agreement
Sprint Planning	Around 20 minutes: agree the goal, available hours, selected issues and ownership
Daily Scrum	Brief check on each agreed project workday, within a 15-minute timebox
Backlog refinement	Around 10 minutes weekly, preferably within an existing work session
Sprint Review	Required five-minute teacher reviews on 17 November and 3 December
Retrospective	Around 10 minutes after each review: record one improvement for the next sprint

Maintain the Product Backlog, Sprint Backlog and working increment throughout. Incomplete work returns to the backlog for reprioritisation; it does not automatically carry forward.

5. Make individual contributions visible on GitHub

Use one public repository and one GitHub Projects board.

Each issue should contain:

- A small user story or technical outcome.
- Acceptance criteria.
- One accountable assignee.
- Sprint, module, priority and estimated effort.
- Dependencies and a link to verification evidence.

Split the work packages above into tasks of approximately **one or two hours**. Use board states **To do** → **In progress** → **In review** → **Done**, with separate backlog and sprint views.

For every contribution:

1. Assign the issue before starting.
2. Create a branch linked to that issue.
3. Commit meaningful progress from the contributor's own account.
4. Reference the issue in commits and the pull request.
5. Obtain a teammate's review before merging.
6. Close the issue only after its acceptance criteria are demonstrated.

Rotate reviews: **Aimar reviews Alessandro, Alessandro reviews Leire, and Leire reviews Aimar**, adjusting where pairing creates a conflict.

Each member should make meaningful commits during every sprint and normally during each week in which they work. Avoid batching all contributions at sprint end. For paired work, include `Co-authored-by:` attribution; each member must also contribute from their own account.

Commit counts help reveal continuity, but contribution quality, completed issues and shared understanding matter more than matching totals.

6. Apply one Definition of Done

An issue is Done when:

- Its acceptance criteria pass and evidence is linked.
- The change is integrated and reviewed by a teammate.
- Relevant automated or hardware checks pass.
- Affected documentation is updated.
- Configuration contains no committed credentials.
- Another member can explain the change.

Hardware claims require checks on the actual Raspberry Pi. Simulated readings remain labelled as simulated.

Proposed validation targets should be agreed during Sprint 1: for example, at least **18 correctly classified crossings out of 20 controlled trials**, plus separate pause and reversal tests. Record the results and limitations rather than presenting an occupancy estimate as an exact count.

A single monitored entrance represents total occupancy only when all access to the test area is controlled and the initial count is known.

7. Protect the final delivery

If work exceeds capacity, remove optional features first, then simplify presentation and activity rules. Preserve the working sensing-to-feedback flow, validation, documentation and contribution evidence.

By **11 December**, prepare:

- Release tag such as `v1.0`, identifying the submitted code.
- Public repository and GitHub Projects links.
- README and wiki covering behaviour, wiring, installation and code structure.
- Half-page joint reflection and individual self-assessment, including the proposed grade.
- `groupXX_Final_Project_IoT.zip` for ALUD.

Commits after the delivery release tag do not count towards evaluation. Rehearse the **15-minute demonstration during the week of 14 December** using the working system, repository and board. All three members must participate and understand every major module.

Immediate action: complete any outstanding repository and board setup, confirm access for all three members and the required teacher collaborator `dieguich`, then turn the Sprint 1 work packages into assigned issues. The brief originally required repository setup on 3 November.